

Overview

In this work, we extend the DiskANN framework by introducing a set of algorithmic improvements aimed at enhancing graph quality, search efficiency, and adaptability. While the original DiskANN constructs a static graph optimized purely on geometric distance, our approach transforms it into a **dynamic, learning-driven system**.

The proposed system integrates three complementary modules:

1. Entropy-Based Edge Pruning
2. Navigation-Aware Search Optimization
3. Adaptive Graph Learning (Self-Refinement)

Together, these modifications enable the graph to not only represent the data distribution more effectively but also evolve based on real query patterns.

1. Entropy-Based Edge Pruning (Person 1)

Motivation

The original α -RNG pruning strategy in Vamana relies solely on distance-based heuristics, which can lead to redundant local connections and insufficient long-range diversity.

Proposed Idea

We introduce an **entropy-inspired diversity metric** to guide edge selection. Instead of pruning based only on proximity, we measure how much *new information* a candidate node contributes relative to already selected neighbors.

Method

For a candidate node c and selected set S :

$$\text{diversity}(c, S) = \min_{n \in S} d(c, n) \quad \text{diversity}(c, S) = \min_{n \in S} d(c, n)$$
$$\text{normalized diversity} = \frac{\text{diversity}(c, S)}{d(p, c)} \quad \text{normalized diversity} = \frac{\text{diversity}(c, S)}{d(p, c)}$$

The pruning threshold is modified as:

$$\alpha_{\text{eff}} = \alpha \cdot (1 + \lambda \cdot \text{normalized diversity}) \quad \alpha_{\text{eff}} = \alpha \cdot (1 + \lambda \cdot \text{normalized diversity})$$

where λ controls the strength of entropy influence.

Impact

- Preserves long-range edges

- Improves graph connectivity
- Reduces local redundancy

2. Navigation-Aware Search Optimization (Person 2)

Motivation

Standard greedy search selects nodes purely based on distance, ignoring how useful a node is for reaching the final nearest neighbors.

Proposed Idea

We introduce **edge usefulness scoring**, allowing the algorithm to learn which edges are more effective during search.

Method

For each edge $(u \rightarrow v)$, we maintain:

- **Traversals(u,v)** $\rightarrow$ number of times edge is visited
- **Helpful(u,v)** $\rightarrow$ number of times it leads to useful candidates

$$\text{usefulness}(u,v) = \frac{\text{Helpful}(u,v)}{\text{Traversals}(u,v)}$$

During search, candidate selection is biased using:

$$d' = d \cdot (1 - \beta \cdot \text{usefulness})$$

where β controls influence of learned navigation.

Impact

- Reduces unnecessary exploration
- Decreases search hops
- Improves latency

3. Adaptive Graph Learning (Person 3 – Your Contribution)

Motivation

DiskANN constructs a static graph that does not evolve with query patterns, limiting its effectiveness in real-world scenarios where query distributions change over time.

Proposed Idea

We introduce a **feedback-driven graph refinement mechanism**, where the graph structure is periodically updated based on learned edge usefulness.

Method

Step 1: Learning from Queries

During each search:

- Traversed edges are recorded
- Edges leading to final candidates are marked as *helpful*

This builds a dynamic estimate of edge quality.

Step 2: Adaptive Thresholding

Instead of fixed thresholds, we use **percentile-based adaptive thresholds**:

- Remove bottom 30% edges
- Boost top 20% edges

Let U be usefulness values of edges:

$T_{\text{remove}} = \text{Percentile}(U, 30\%)$
 $T_{\text{boost}} = \text{Percentile}(U, 80\%)$

Step 3: Graph Refinement

Edges are updated as:

- If $\text{usefulness} < T_{\text{remove}}$ → removed
- If $\text{usefulness} > T_{\text{boost}}$ → reinforced

Additionally, neighbors are sorted based on usefulness to prioritize stronger edges.

Learning Loop

Search → Update Scores → Refine Graph → Improved Search
Search → Update Scores → Refine Graph → Improved Search

Impact

- Graph evolves with usage
- Removes ineffective edges
- Strengthens important paths

- Enables long-term performance improvement

Combined System Effect

The integration of all three components results in a system that is:

Property	Original DiskANN	Proposed System
Graph Type	Static	Adaptive
Edge Selection	Distance-based	Information-driven
Search Strategy	Greedy	Learned + biased
Learning	None	Continuous
Optimization	Local	Global + dynamic

Expected Performance Improvements

- Reduced search hops
- Lower latency
- Improved recall stability
- Better performance over time

Key Insight

“We transform ANN search from a static geometric problem into a dynamic, learning-driven system that improves with usage.”

Person 3 work :

While prior improvements (entropy-based pruning and navigation-aware search) enhance graph construction and traversal, the system still remains **fundamentally static** after index construction. In real-world scenarios, query distributions are non-uniform and evolve over time, making static graph structures suboptimal.

To address this limitation, we introduce a **self-adaptive learning layer** that enables the graph to continuously improve based on observed query behavior.

Core Contribution

We design and implement a **feedback-driven graph refinement mechanism** that dynamically updates graph connectivity using learned edge usefulness, transforming DiskANN into a **self-improving ANN system**.

Detailed Implementation

1. Edge-Level Learning Signal

We leverage the navigation statistics introduced in the system and extend them into a learning signal.

For each directed edge $(u \rightarrow v)$, we track:

- $\text{Traversals}(u,v)$: number of times edge is explored
- $\text{Helpful}(u,v)$: number of times it contributes to final candidate set

The **edge usefulness score** is defined as:

$$\text{usefulness}(u,v) = \frac{\text{Helpful}(u,v)}{\text{Traversals}(u,v)} \quad \text{usefulness}(u,v) = \text{Traversals}(u,v) \cdot \text{Helpful}(u,v)$$

This score acts as a **data-driven estimate of edge quality**.

2. Learning from Query Execution

During each search operation:

- All traversed edges are recorded
- A subset of nodes (final candidate set) is marked as *useful*
- Edge statistics are updated accordingly

This creates a **continuous feedback loop** between search behavior and graph structure.

3. Periodic Graph Refinement

A new function `refine_graph()` is introduced and triggered periodically (every 1000 queries):

```
if (total_queries_ % 1000 == 0) {
    refine_graph();
}
```

This ensures:

- Learning is **incremental**
- Overhead is controlled
- Graph updates are **batched and stable**

4. Adaptive Thresholding (Key Innovation)

Instead of using fixed thresholds, we implement a **percentile-based adaptive thresholding mechanism**, making the system robust to varying data distributions.

For each node u , we compute usefulness values of its outgoing edges:

$$U = \{\text{usefulness}(u,v) \mid v \in N(u)\} \quad U = \{\text{usefulness}(u,v) \mid v \in N(u)\}$$

Thresholds are defined as:

$$T_{\text{remove}} = \text{Percentile}(U, 30\%) \quad T_{\text{boost}} = \text{Percentile}(U, 80\%)$$

Interpretation:

- Bottom 30% edges → considered weak → removed

- Top 20% edges → considered strong → reinforced

5. Graph Structure Update

The graph is updated as follows:

Edge Removal

Edges with usefulness below T_{remove} are pruned:

$\text{usefulness}(u,v) < T_{\text{remove}} \Rightarrow \text{remove edge}$
 $\text{usefulness}(u,v) < T_{\text{remove}} \Rightarrow \text{remove edge}$

Edge Reinforcement

Edges above T_{boost} are strengthened by duplication:

$\text{usefulness}(u,v) > T_{\text{boost}} \Rightarrow \text{reinforce edge}$
 $\text{usefulness}(u,v) > T_{\text{boost}} \Rightarrow \text{reinforce edge}$

Neighbor Reordering

Remaining neighbors are sorted by usefulness:

$\text{order}(v) \propto \text{usefulness}(u,v)$
 $\text{order}(v) \propto \text{usefulness}(u,v)$

This ensures that **high-quality edges are explored earlier during search**.

6. Self-Improving Learning Loop

The overall system now operates as:

$\text{Search} \rightarrow \text{Edge Updates} \rightarrow \text{Graph Refinement} \rightarrow \text{Improved Search}$
 $\text{Search} \rightarrow \text{Edge Updates} \rightarrow \text{Graph Refinement} \rightarrow \text{Improved Search}$

This creates a **closed-loop adaptive system**.

Key Improvements Introduced

Dynamic Graph Evolution

- Graph structure is no longer fixed
- Continuously adapts to query patterns

Data-Driven Optimization

- No manual tuning required
- Thresholds adapt automatically

Improved Search Efficiency

- Fewer hops
- Reduced distance computations
- Faster convergence

Robustness

- Handles skewed and evolving query distributions

Practical Impact

Over multiple queries, the system exhibits:

- Progressive reduction in traversal hops
- Stabilization of latency
- Improved recall consistency
- Better utilization of graph structure

Key Insight (Highlight This Line)

“We transform DiskANN from a static index into a self-optimizing system that continuously refines its structure based on observed query behavior.”

Summary of Your Contribution

Component	Contribution
Learning Mechanism	Designed edge usefulness-based learning
Graph Refinement	Implemented dynamic restructuring
Thresholding	Introduced percentile-based adaptive thresholds
Integration	Connected learning with search pipeline
System Impact	Enabled self-improving ANN graph

Conclusion

This work demonstrates that incorporating learning mechanisms into graph-based ANN systems can significantly enhance their efficiency and adaptability. By combining entropy-based pruning, navigation-aware search, and adaptive graph refinement, we move beyond traditional static indexing toward **self-optimizing ANN systems suitable for real-world deployment**.